

Why DevOps?

From Delivery Pain to Continuous Flow

Building on your SDLC, Jira, Scrum, and Infrastructure knowledge — using MTS as our shared story.

QUICK RECAP

You Already Know the SDLC



Every software delivery follows this journey — from business need to live system. MTS followed exactly this path.

So... If SDLC Exists, Why DevOps?

Good question.

The stages exist. The teams exist. The process exists.

Then why does delivery still feel painful?

MTS STORY BEGINS

A Simple Change in MTS

The Change

- Developer fixes **receiver validation** logic in MTS
- Validates that receiver account numbers meet new format rules
- Small change — a few lines of code

Expected Outcome

- Invalid receivers get rejected cleanly
- Transfers route correctly
- Customer experience improves

📝 Sounds straightforward. Let's see what actually happens.

"Works on My Machine"



Local

The developer runs the fix. Tests pass. Everything looks great — on their laptop.

- Local environment has different config than QA or Production
- No automated way to share a verified build
- The fix exists only on one machine

QA Asks: "Where Is My Build?"

Developer

"I pushed the code. Check the repo — or maybe I emailed it."

QA Engineer

"I need a deployable build on the test environment. Who sets that up?"

Result

Hours lost. Testing blocked. The clock is ticking toward the release date.

Ops Asks: "What Exactly Changed?"

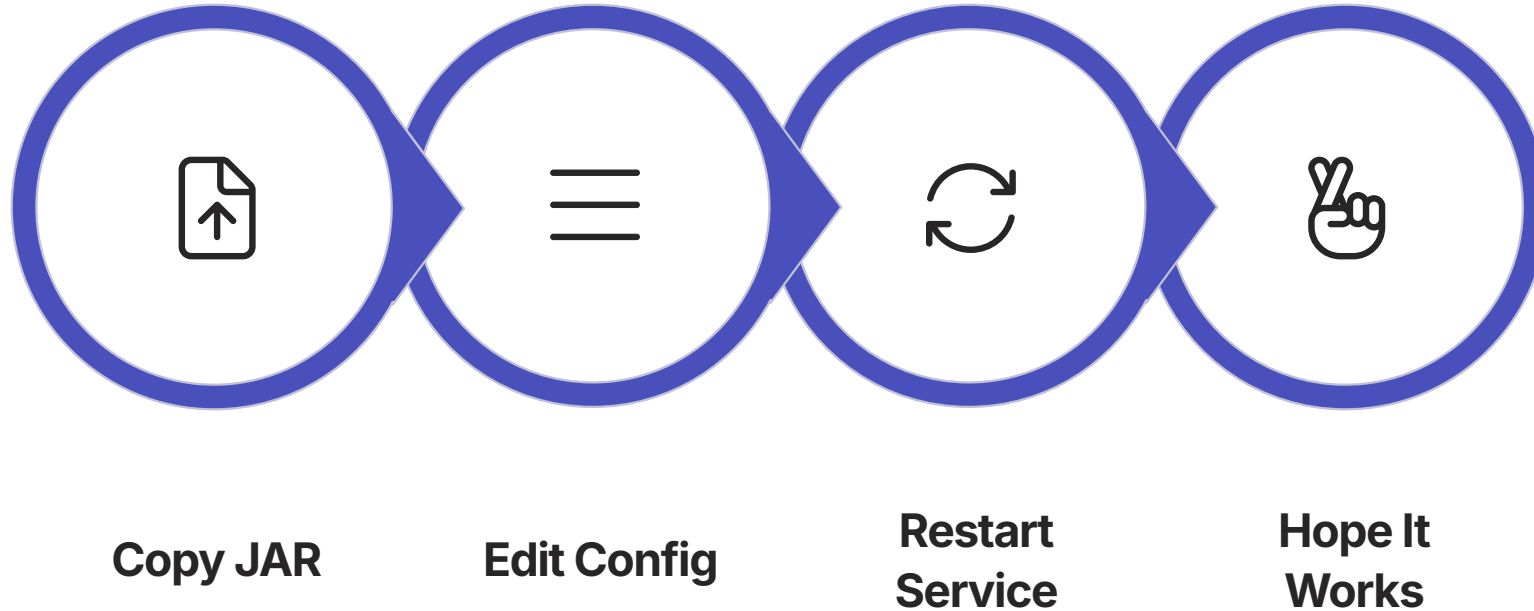
Ops Team Needs to Know

- Which files changed?
- Are there new config values?
- Any new dependencies or env variables?
- What should we roll back to if it breaks?

What They Received

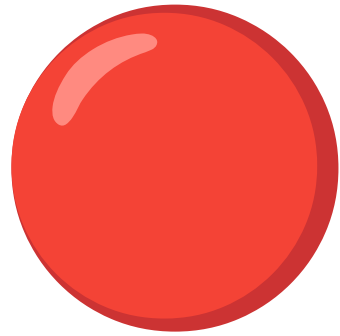
- An email with a zip file
- Verbal instructions: "just copy the JAR"
- No formal change record

Manual Steps + Config Mismatch



- Production config had different DB connection string than QA
- One step missed — no checklist, no automation
- Nobody noticed until the service started

Production Fails



DOWN

MTS transfers are failing. Customers cannot send money.

- Alerts fire — but nobody knows which change caused it
- Rollback is also manual and risky
- The on-call phone starts ringing

Three Teams. One Customer Impact.

Dev says

"Code was fine. It worked in my environment."

QA says

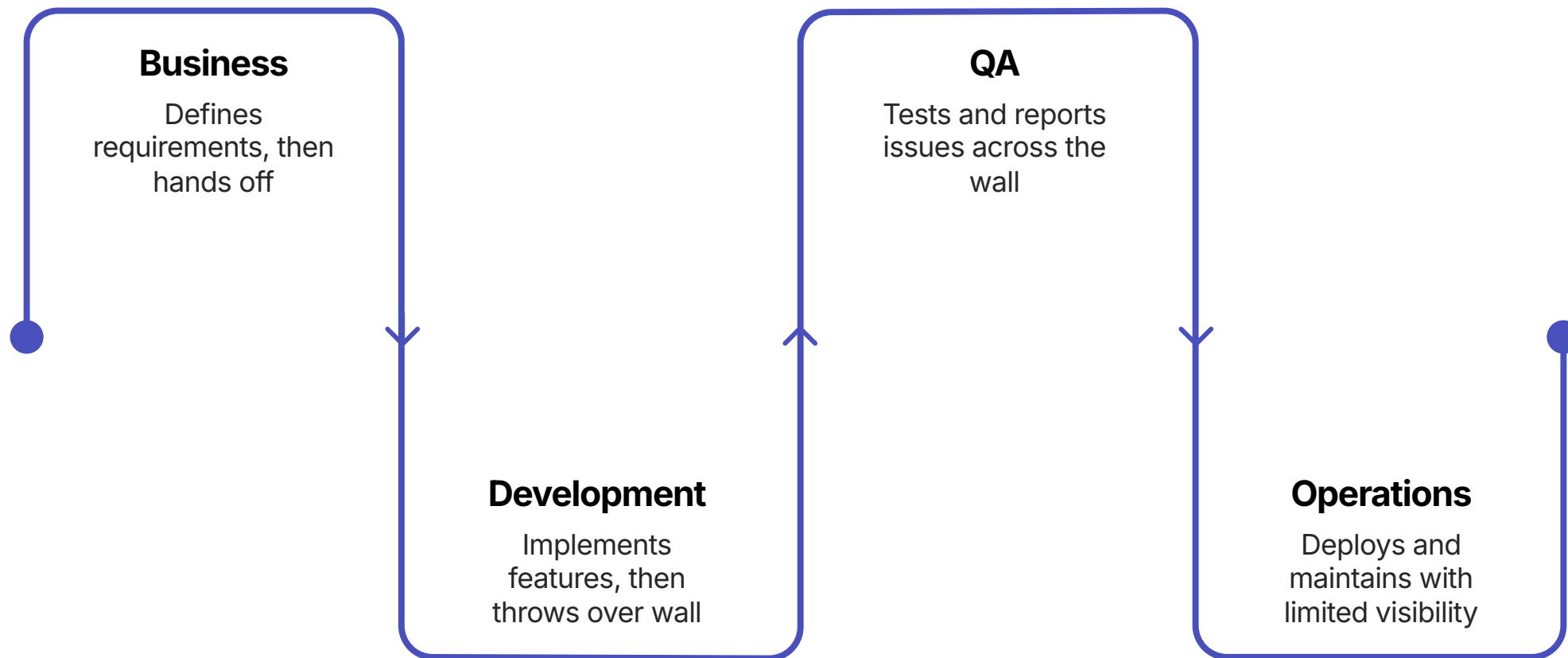
"We never got enough time to test properly."

Ops says

"The build we received was incomplete. This isn't on us."

Meanwhile — the customer just wants their transfer to go through.

The Traditional Silo Structure



Each team finishes their part and **throws it over the wall**. No shared ownership. No shared visibility.

Dev's Goal: Change Quickly

Dev Team Optimizes For

- Shipping features fast
- Responding to business requests
- Frequent code commits
- Moving on to the next story

Success Looks Like

- Stories marked "Done" in Jira
- New features in the build
- Code reviewed and merged

Ops' Goal: Keep Production Stable

Ops Team Optimizes For

- Uptime and reliability
- Controlled, low-risk changes
- Predictable, tested deployments
- Clear rollback paths

Success Looks Like

- Zero production incidents
- Stable, well-monitored systems
- Fewer emergency calls at night

QA's Goal: Protect Quality

QA Team Optimizes For

- Thorough test coverage
- Catching defects before production
- Adequate time to verify
- Clear requirements to test against

Success Looks Like

- No bugs escaping to production
- Stable test environments
- Well-documented test results

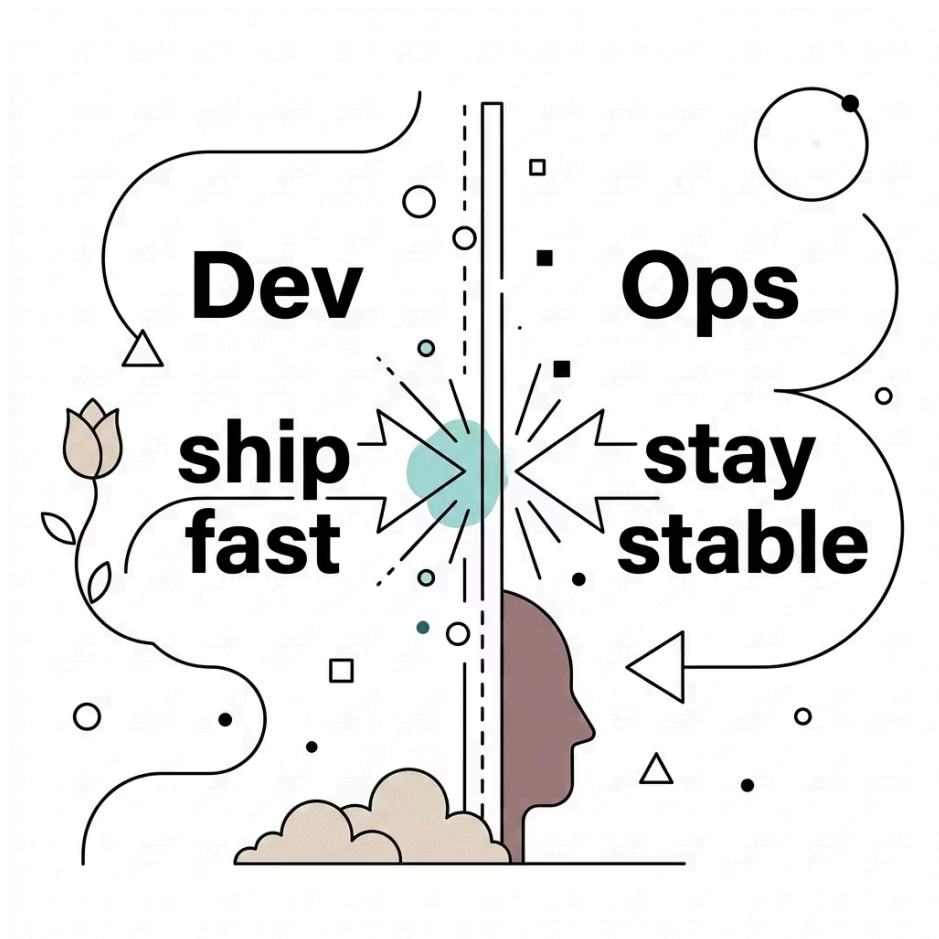
None of These Goals Are Wrong

But optimizing separately creates friction.

- Dev pushes change → Ops slows it down for safety
- QA blocks release → Dev feels bottlenecked
- Ops changes config → QA's tests become invalid

Each team is doing the right thing — for their own silo.

The Wall of Confusion



What the Wall Represents

- Different metrics, different incentives
- No shared view of delivery health
- Communication only at handoff points
- Blame flows across the wall when things break

SYMPTOMS

Long Handoffs. Manual Work. Late Feedback.



Long Handoffs

Days or weeks pass between stages. Work sits idle waiting for the next team.



Manual Work

Builds, deploys, and tests done by hand. Different results every time.



Late Feedback

Bugs found in production — days after the code was written.

MORE SYMPTOMS

Big Releases. Release Fear. Blame.



Big Releases

Months of changes bundled together — impossible to isolate what broke.



Release Fear

Everyone dreads release night. "Do not release on Fridays."



Blame Culture

When it fails, teams point fingers. Root cause stays hidden.

What If Delivery Became One Shared Flow?

What if the wall disappeared? 🤝

- All teams work toward the same delivery outcome
- Feedback arrives early — not at release night
- Automation handles the repeatable steps
- Small changes, released often, with confidence

DevOps = Culture + Collaboration + Practices + Automation + Feedback



Culture

Shared ownership. Psychological safety.
Learning over blame.



Collaboration

Dev, QA, and Ops working together —
not in sequence.



Practices

Small batches, quality gates, standard
workflows.



Automation

Consistent, repeatable builds, tests, and deployments.



Feedback

Fast signals from pipeline and production — back to the team.

DevOps Does NOT Replace SDLC

SDLC Still Exists

- Requirements still get gathered
- Design still happens
- Testing is still essential
- Operations still run production

What DevOps Adds

- Faster flow between stages
- Automation to reduce manual gaps
- Feedback loops across all stages
- Shared ownership of the outcome

 DevOps accelerates and connects SDLC stages — it does not remove them.

DevOps Does NOT Mean One Person Does Everything

Common Misconception

"We hired a DevOps engineer. They will do Dev, QA, and Ops now."

Reality

DevOps is a **team culture and way of working** — not a single job role that absorbs other roles.

What Changes

How teams **collaborate, share responsibility, and use automation** — not who does every task.

DevOps Does NOT Mean "Just Tools"

Tools Are Enablers

- Jenkins, Docker, Kubernetes, Terraform — useful tools
- But tools without culture = faster broken process
- Buying tools is not the same as adopting DevOps

Culture Comes First

- Shared ownership of delivery
- Willingness to collaborate early
- Commitment to learning from failure

📌 We will learn the tools in later decks — after we understand the *why*.

Shared Ownership of the Delivery Outcome

We ship it together. We own it together.

- A defect in production is a team problem — not Ops' problem alone
- A slow pipeline affects everyone — not just Dev
- Quality is everyone's responsibility — not only QA's

"You Build It" — With Operational Awareness

What This Means

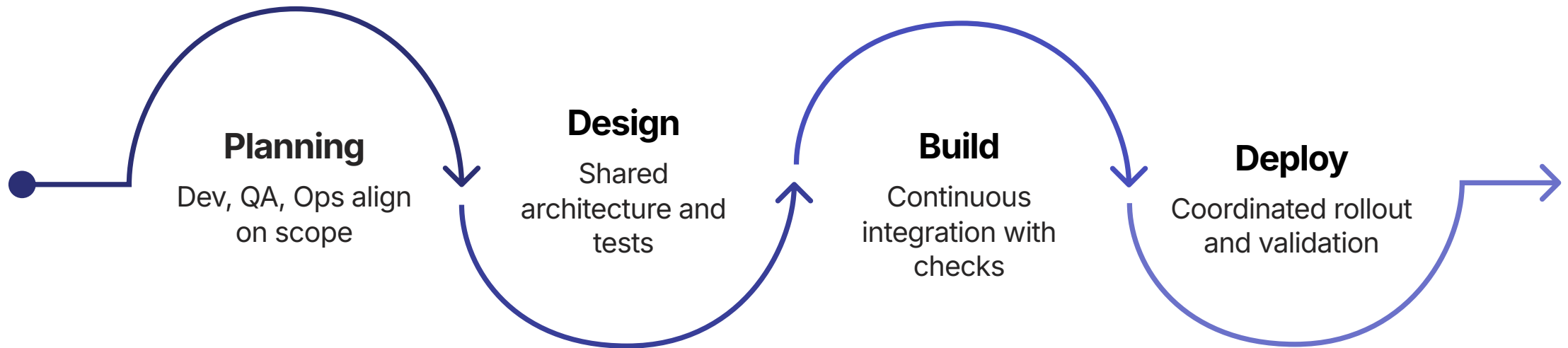
- Developers care about how code runs in production
- Teams consider monitoring, logging, and failure modes — not just features
- Operational concerns are part of the definition of "done"

What This Does NOT Mean

- One developer owns everything end-to-end
- Ops team disappears
- QA role is eliminated

COLLABORATION

Earlier Together — Not Only at Release Time



When all three teams align early, surprises at release night shrink dramatically.

Small Changes Reduce Risk

Big Releases

- Months of changes bundled
- Hard to find root cause when it breaks
- High-stakes, high-fear deployments

Small, Frequent Changes

- One change at a time
- Easy to identify what broke
- Low-risk, routine deployments

📌 MTS receiver validation fix = perfect example of a small, scoped change.

Fast Feedback Finds Problems Early

Slow Feedback

1. Bug introduced in code
2. Weeks pass through manual QA and release
3. Bug found in production
4. Expensive, stressful fix

Cost: High

Fast Feedback

1. Bug introduced in code
2. CI pipeline runs within minutes
3. Test fails → developer notified immediately
4. Fix is quick and cheap

Cost: Low

The closer feedback is to when the code was written, the cheaper and faster the fix.

Automation Makes Repeatable Work Consistent

1

Manual

Different result every time. Human error. Hard to audit.

2

Automated

Same steps, same order, every time.
Logged and traceable.

3

Confident

Teams trust the pipeline. Focus shifts to higher-value work.

Production Feedback Comes Back to the Team

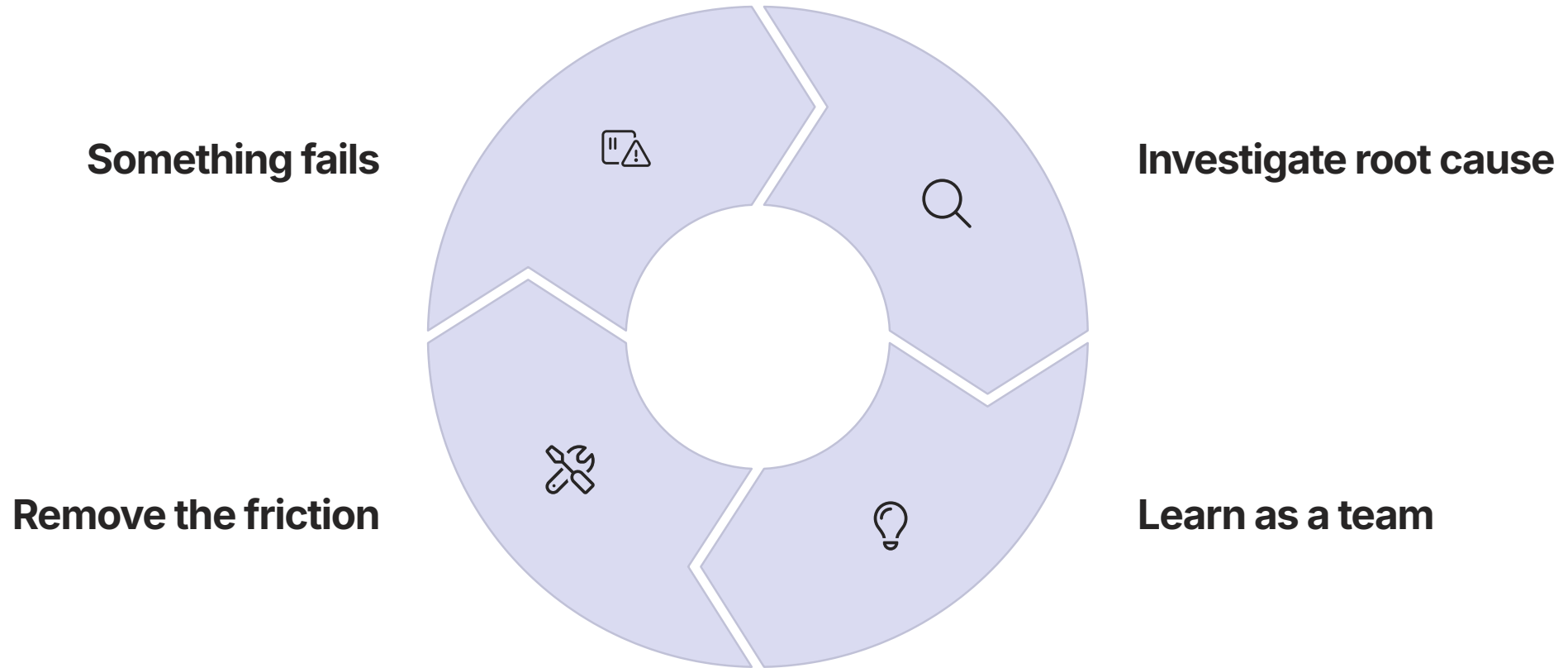
Without Monitoring

- Teams guess what's happening in production
- Problems found by customers, not engineers
- Fixes are reactive and slow

With Observability

- Dashboards show system health in real time
- Alerts reach the right team immediately
- Data guides the next improvement

Continuous Improvement: Learn, Don't Blame



Every failure is a signal — not a verdict on a person.

BEFORE VS. AFTER

Traditional Delivery vs. DevOps Delivery

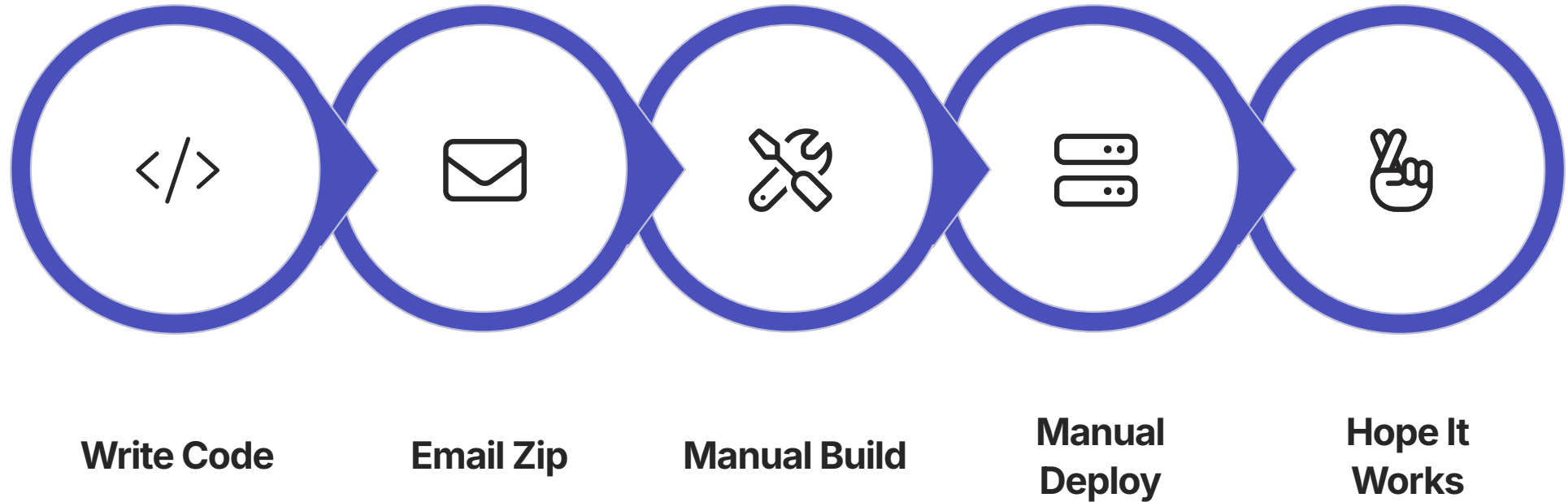
Before DevOps

- Long release cycles (months)
- Manual, error-prone handoffs
- Late testing — bugs found in production
- Release fear and blame culture
- Teams work in isolation

With DevOps

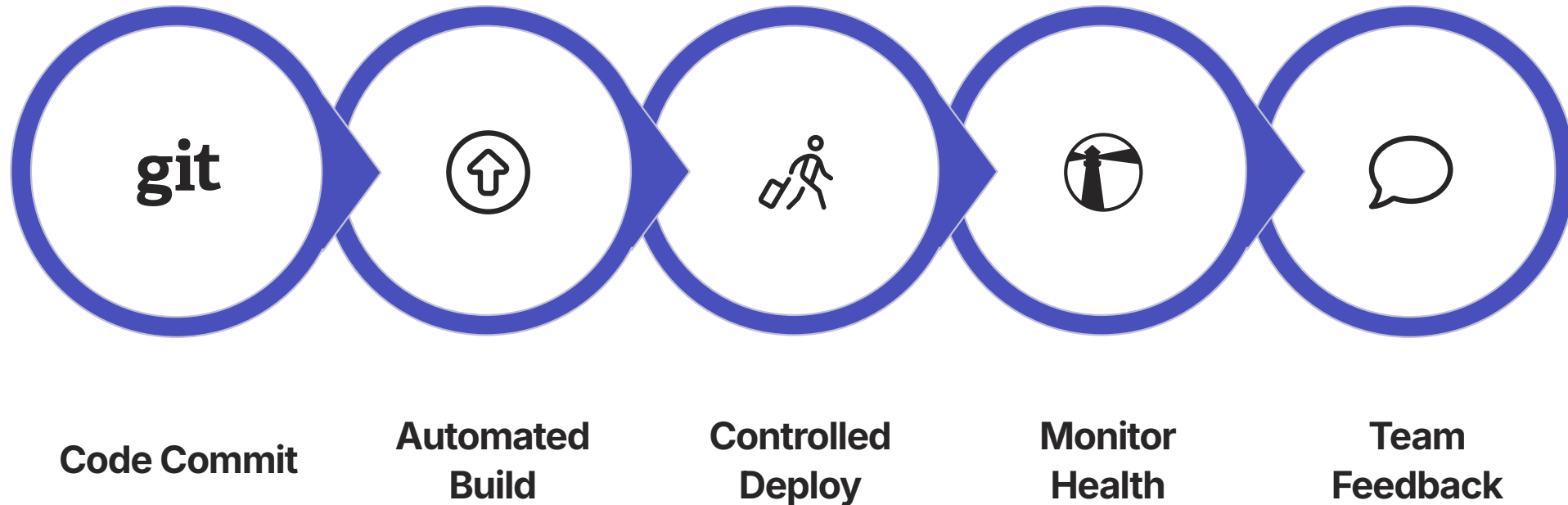
- Frequent, small releases
- Automated, consistent pipelines
- Testing integrated throughout
- Confidence and shared ownership
- Teams collaborate continuously

MTS Delivery the Old Way



- No traceability — what version is in production?
- No consistency — every deployment slightly different
- No early feedback — problems surface at release night

MTS Delivery with DevOps Practices



- Every step is traceable and repeatable
- Problems surface in the pipeline — not in production
- Teams see what's happening at every stage

What Does "Continuous" Really Mean?

Does it mean every second? 🤔

Let's clear this up before we go further.

Continuous ≠ Every Second

What It Does Mean

- **Frequent** — not once a quarter
- **Repeatable** — same process every time
- **Enabled by automation** — not dependent on heroics
- **Guided by feedback** — each step informs the next

What It Does NOT Mean

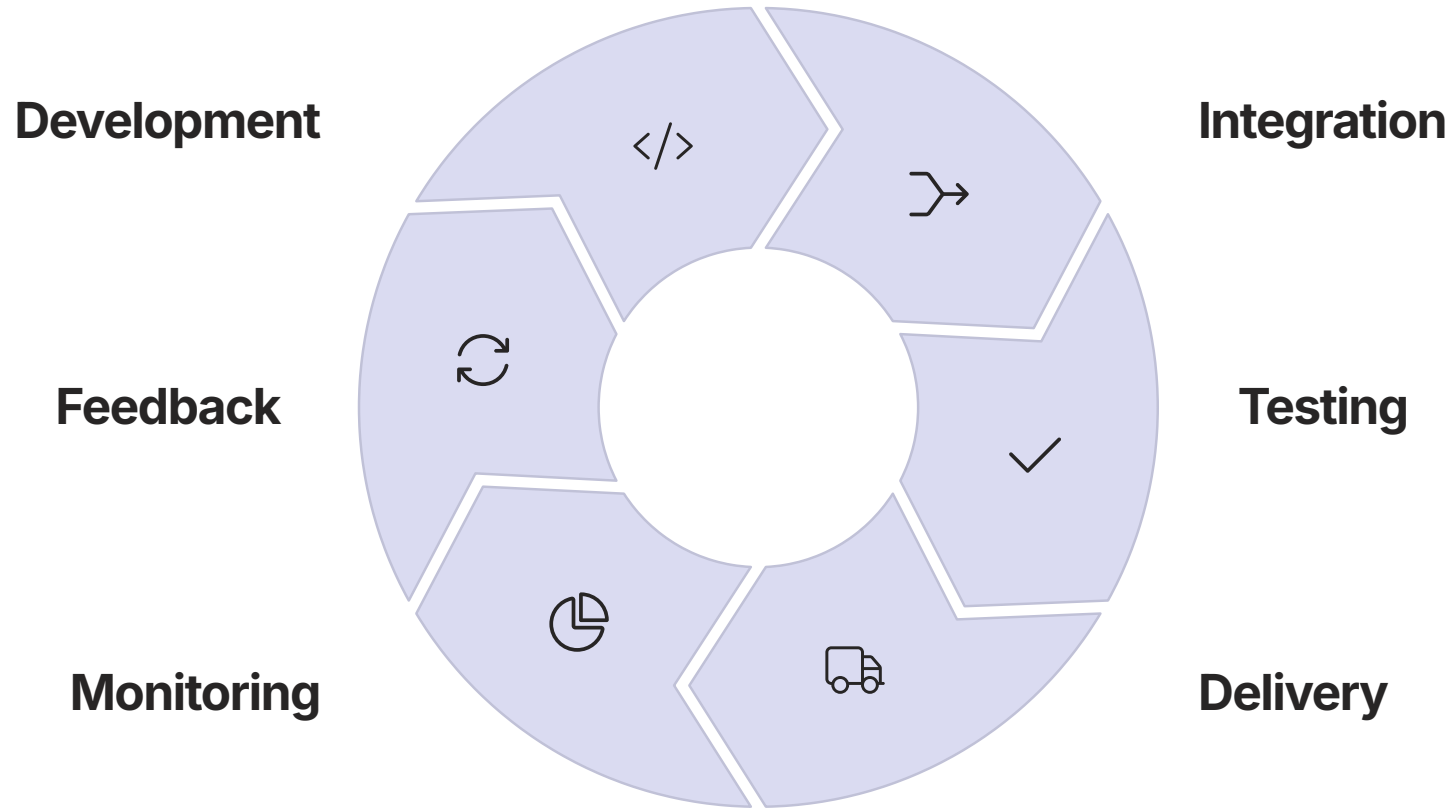
- Deploying every second
- No human decisions or approvals
- Non-stop activity with no planning

Introducing the 7 C's of DevOps

 The "7 C's" is a **useful teaching model** — not a universal industry standard. Different organizations and frameworks use different lists. We use it here to structure our learning.

- A way to organize the key practices of a DevOps pipeline
- Each "C" represents a continuous practice or phase
- They connect and reinforce each other in a loop

The 7 C's as a Loop



+ **Continuous Operations / Improvement** — maintaining service and improving delivery over time.

C1

Continuous Development

What It Means

- Plan and write code in **small, focused increments**
- Use version control (Git) to manage changes continuously
- Work from a Jira story — scoped and well-defined

Why It Matters

- Small changes are easier to review and test
- Reduces the risk of large, hard-to-debug releases
- Keeps work aligned to business value

MTS EXAMPLE

Receiver Validation = A Continuous Development Win

The Jira Story

"As a transfer system, I should reject invalid receiver account formats with a clear error."

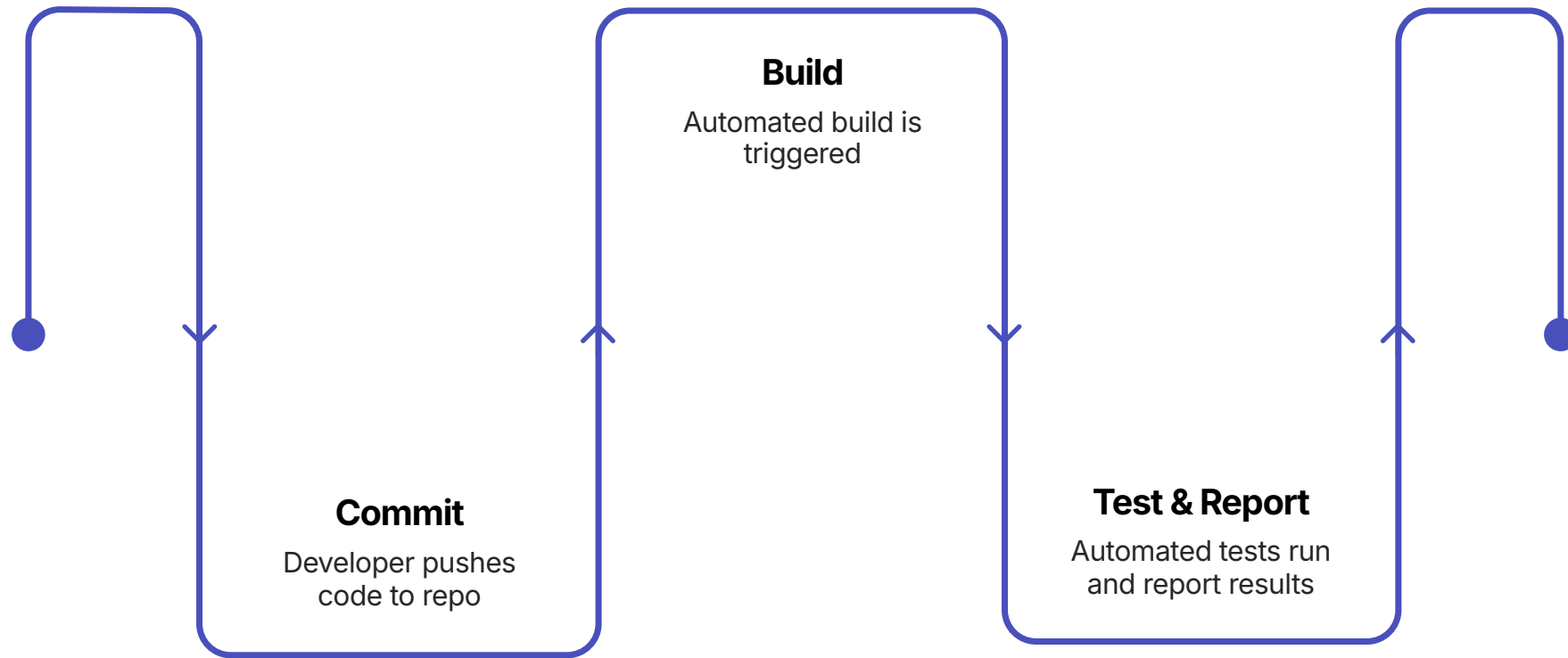
The Change

A small, scoped code change to the validation module. One responsibility. One purpose.

The Discipline

Not bundled with 20 other changes. Committed to Git with a clear message. Ready to integrate.

Continuous Integration



- Changes from all developers are integrated **frequently** — not saved up
- Automated verification catches conflicts and failures early
- The team always has a known, tested state of the codebase

C3

Continuous Testing

What It Means

- Automated tests run at every stage of the pipeline
- Unit tests → Integration tests → Regression tests
- Fast feedback: pass or fail, immediately after commit

Key Benefit

- Defects found in CI — not in production
- QA focus shifts to exploratory and edge-case testing
- Quality is built in — not checked at the end

Continuous Delivery

What It Means

- Software is always in a **releasable state**
- Automated pipeline builds, tests, and prepares every change
- Production release is a **deliberate business decision** — made by a human

What It Does NOT Mean

- Automatic push to production without approval
- No change management or controls
- Humans are removed from the release decision



Continuous Delivery = always *ready* to deploy. The release decision still belongs to the team.

Continuous Delivery vs. Continuous Deployment

Continuous Delivery

- Pipeline prepares a deployable artifact
- **Human decides** when to release to production
- Release is controlled and intentional
- Common in regulated environments (e.g., banking, healthcare)

Continuous Deployment

- Every qualifying change is **automatically deployed** to production
- No manual release gate after automated tests pass
- Requires very high confidence in automated test coverage
- Common in high-velocity web/SaaS products

C5

Continuous Monitoring



Application Health

Response times, error rates, transaction success — is MTS processing transfers correctly?



Infrastructure Health

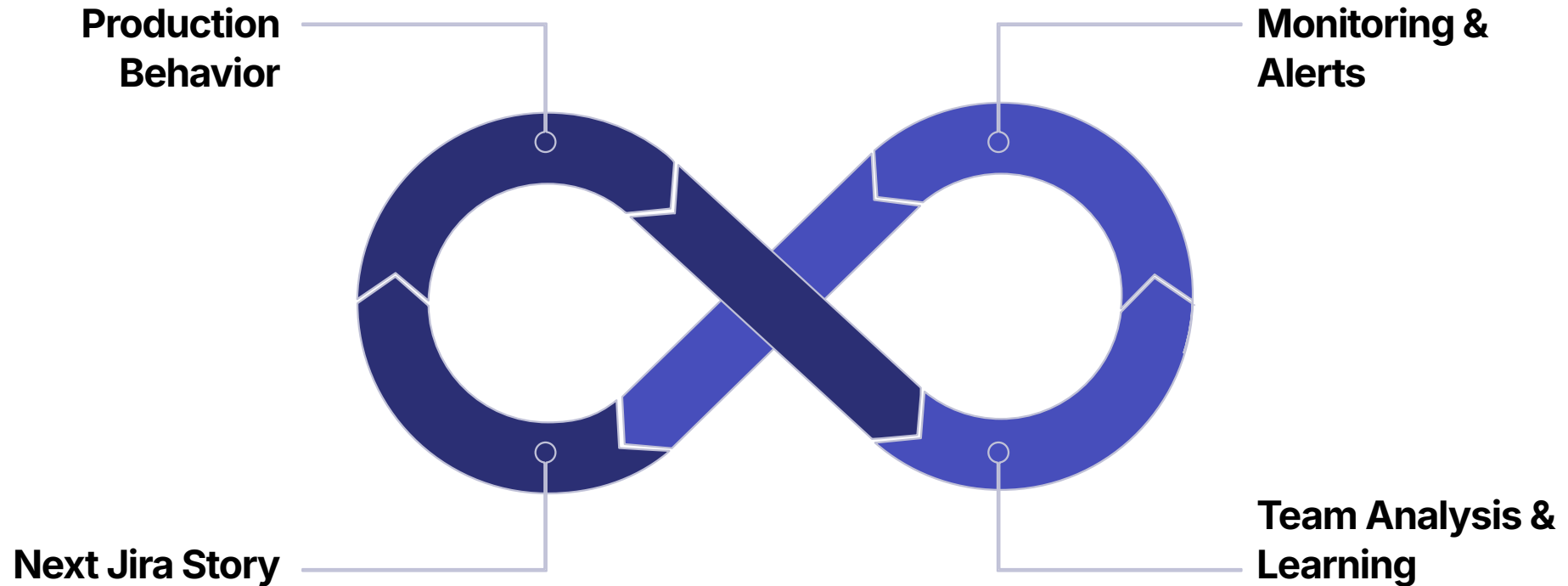
CPU, memory, disk, network — are the systems MTS runs on healthy?



Alerts & Dashboards

Anomalies surface immediately — teams act before customers notice.

Continuous Feedback



- Production data shapes the next sprint backlog
- Pipeline failures are feedback — not failures of a person
- User behavior informs feature priorities

Continuous Operations / Improvement

Continuous Operations

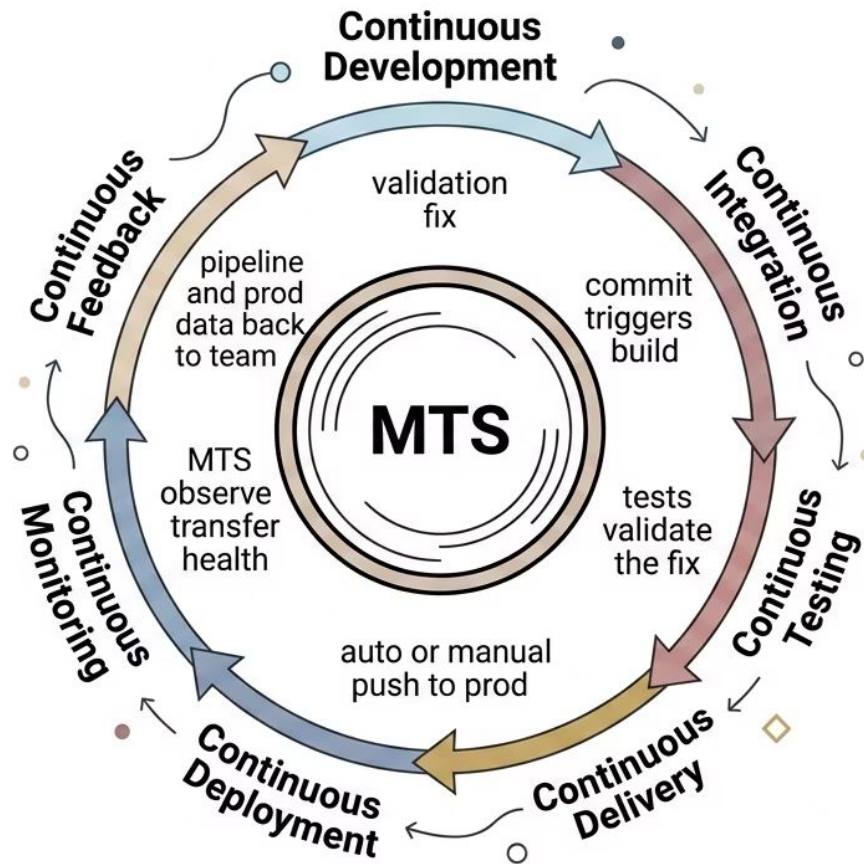
- Keep services running reliably
- Minimize downtime and disruption
- Manage incidents with speed and clarity

Continuous Improvement

- Review what slowed delivery down
- Remove recurring friction from the process
- Make the next release easier than the last

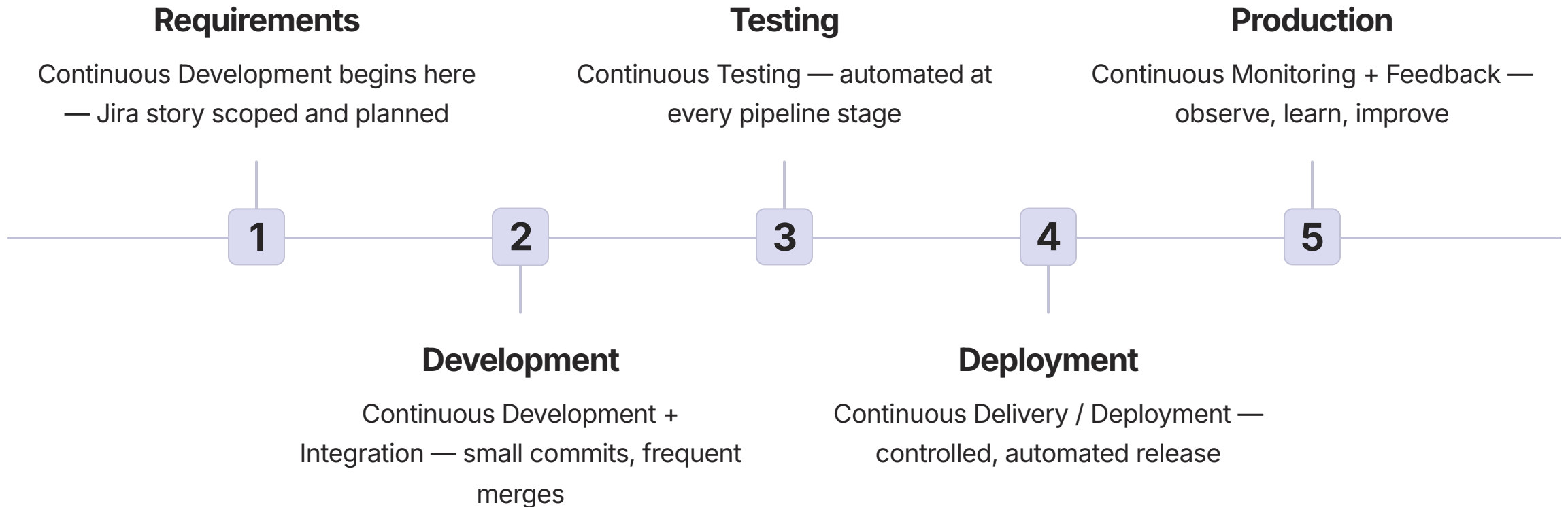
📌 Terminology varies — some frameworks call this "Continuous Improvement" or combine it with operations. The intent is the same: never stop getting better.

All 7 C's Around MTS



Every stage feeds the next. MTS delivery becomes a living, breathing loop — not a one-time event.

7 C's Map to SDLC — They Don't Replace It



Requirements, design, testing, and operations still happen — DevOps accelerates the connections between them.

CONNECTION

Jira in the DevOps World

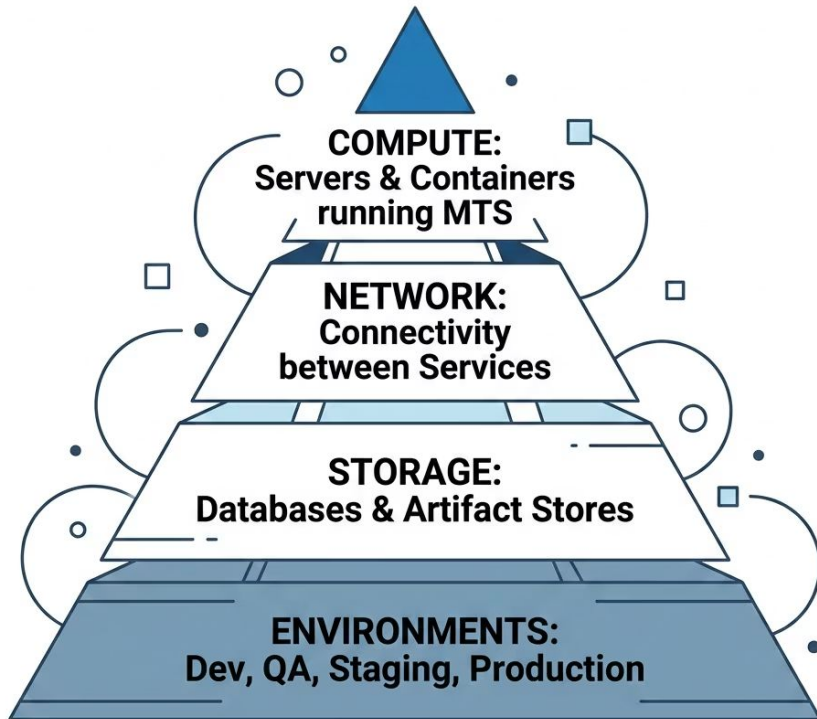
Jira Tracks Work

- Backlog → Sprint → Story → Subtask
- Defects and bugs raised against specific changes
- Acceptance criteria define when "done" means done
- Pipeline status and test results can link back to the story

Pipeline Executes Delivery

- A commit references the Jira story number
- CI pipeline builds and tests automatically
- Deployment tracked — which story is in which environment?
- Feedback loops back to Jira as a comment, bug, or new story

Infrastructure in the DevOps World



Infrastructure Is Where Delivery Runs

- Environments must be consistent — config drift causes failures
- Infrastructure changes follow the same pipeline discipline as code
- DevOps teams care about the health of the infrastructure, not just the application
- Infrastructure as Code (IaC) — covered in later decks

PEOPLE

People Dimension: Collaboration and Psychological Safety



Collaborate Early

Dev, QA, and Ops align from the start of a story — not just at release time.



Psychological Safety

People raise problems early when they trust they won't be blamed for finding them.



Learning Culture

Blameless post-mortems. Incidents are learning opportunities — not performance reviews.

Process Dimension: Smaller Batches, Standard Flow, Quality Gates

1

Smaller Batches

One story at a time. One change per commit. Less risk, faster feedback.

2

Standard Workflow

Everyone follows the same pipeline. No special exceptions for "urgent" releases.

3

Quality Gates

Automated checks at each stage — code won't advance if it fails the gate.

Technology Dimension: The DevOps Toolchain



Source Control (SCM)

Git — tracks every change, who made it, and when.



CI / Automation

Jenkins, GitHub Actions — trigger builds and tests on every commit.



Artifact Repository

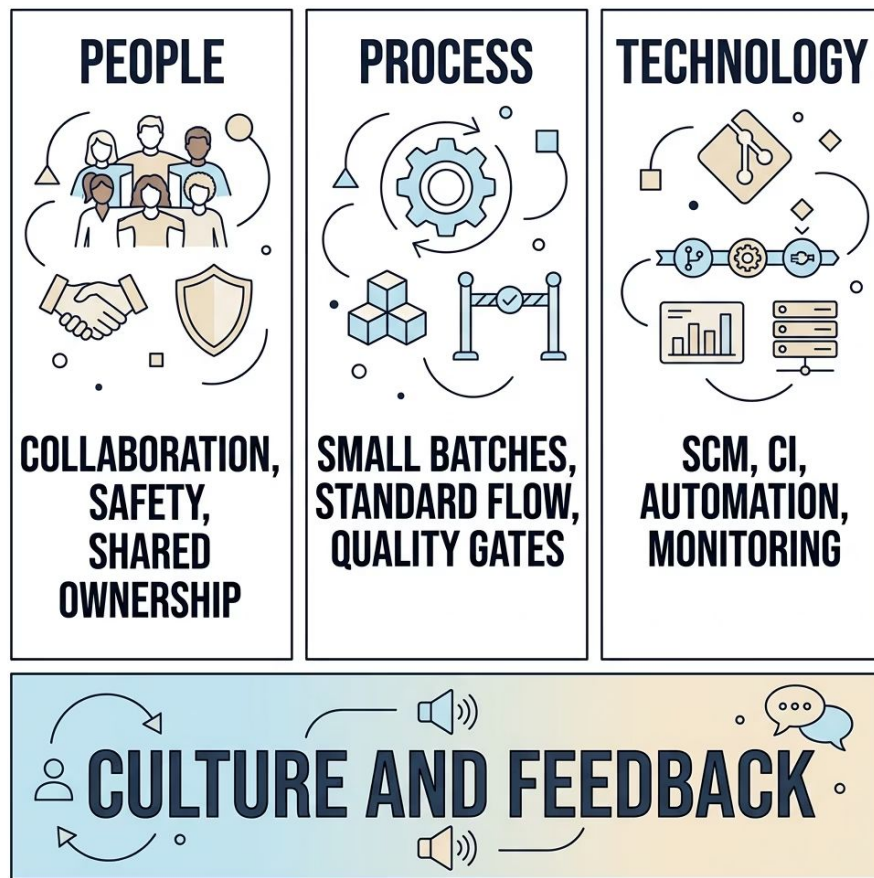
Store tested, versioned builds — ready to deploy to any environment.



Monitoring

Dashboards and alerts — observe MTS in production continuously.

DevOps = People + Process + Technology



No Pillar Works Alone

- **People** without process = good intentions, inconsistent results
- **Process** without technology = slow, manual execution
- **Technology** without people = tools collecting dust

Culture and feedback run **across all three** — they are the glue.

ANTI-PATTERN

Buying Tools Without Changing Handoffs

What It Looks Like

- Leadership buys Jenkins, Kubernetes, and Terraform
- Teams still work in silos with the same handoff delays
- Tools sit unused or are used inconsistently

Why It Fails

- Tools don't fix a collaboration problem
- Automation without ownership = faster confusion
- The wall of confusion stays — it just has a CI server in front of it

 Tools amplify the process you already have — broken or not.

ANTI-PATTERN

Automating a Broken Process

The Temptation

"Our manual deployment is slow and error-prone. Let's script it."

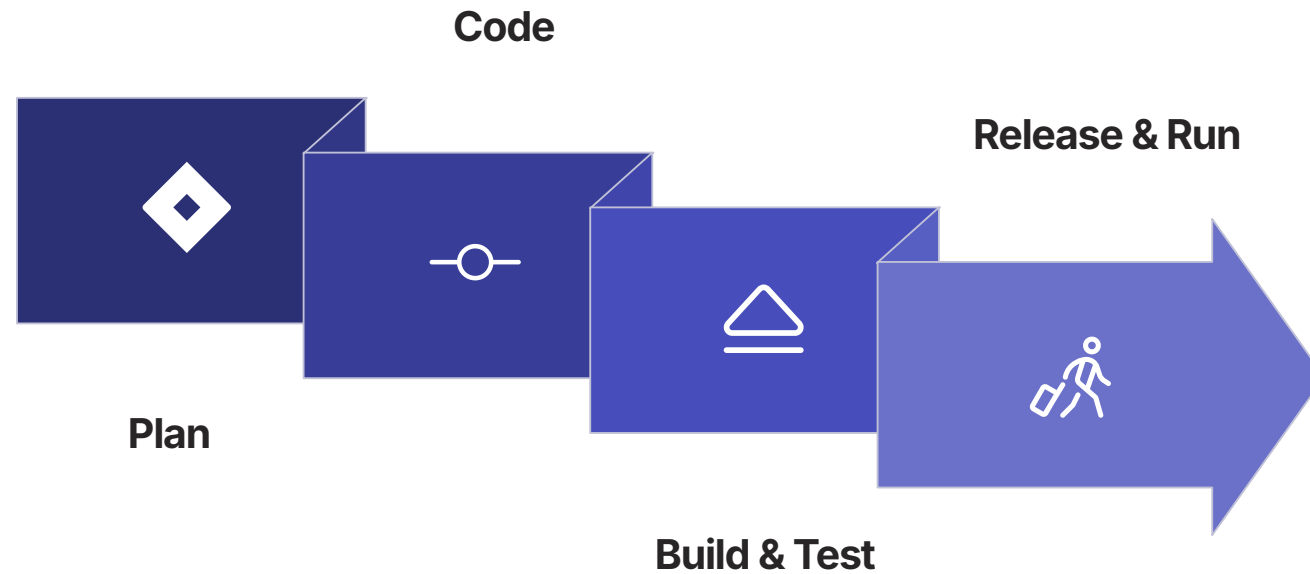
The Problem

If the deployment process is flawed — with missing checks, wrong configs, unclear steps — automation makes the flaws run **faster and more consistently**.

The Right Approach

Understand the process first. Fix the gaps. *Then* automate what is known to be correct.

MTS Full DevOps Journey — Preview



This is the full pipeline we will build knowledge for — piece by piece — across Decks 2 and 3.

Let's Check Understanding

1

Why DevOps?

What delivery problem does DevOps solve? Describe it using the MTS story.

2

Culture vs. Tools

Can a team adopt DevOps by buying Jenkins and Docker alone? Why or why not?

3

Delivery vs. Deployment

What is the key difference between Continuous Delivery and Continuous Deployment?

4

SDLC + DevOps

Does DevOps eliminate the need for requirements, design, or testing stages?

The Developer Has Changed MTS Code. What Happens Next?

Where should that code live?
How do we verify every change automatically?

1

Source Control

Git basics — branching, commits, pull requests

2

CI Pipeline

Automated build and test on every commit

3

Automation

Making delivery consistent, traceable, and fast

Deck 2: SCM, Git Concepts, CI and Automation